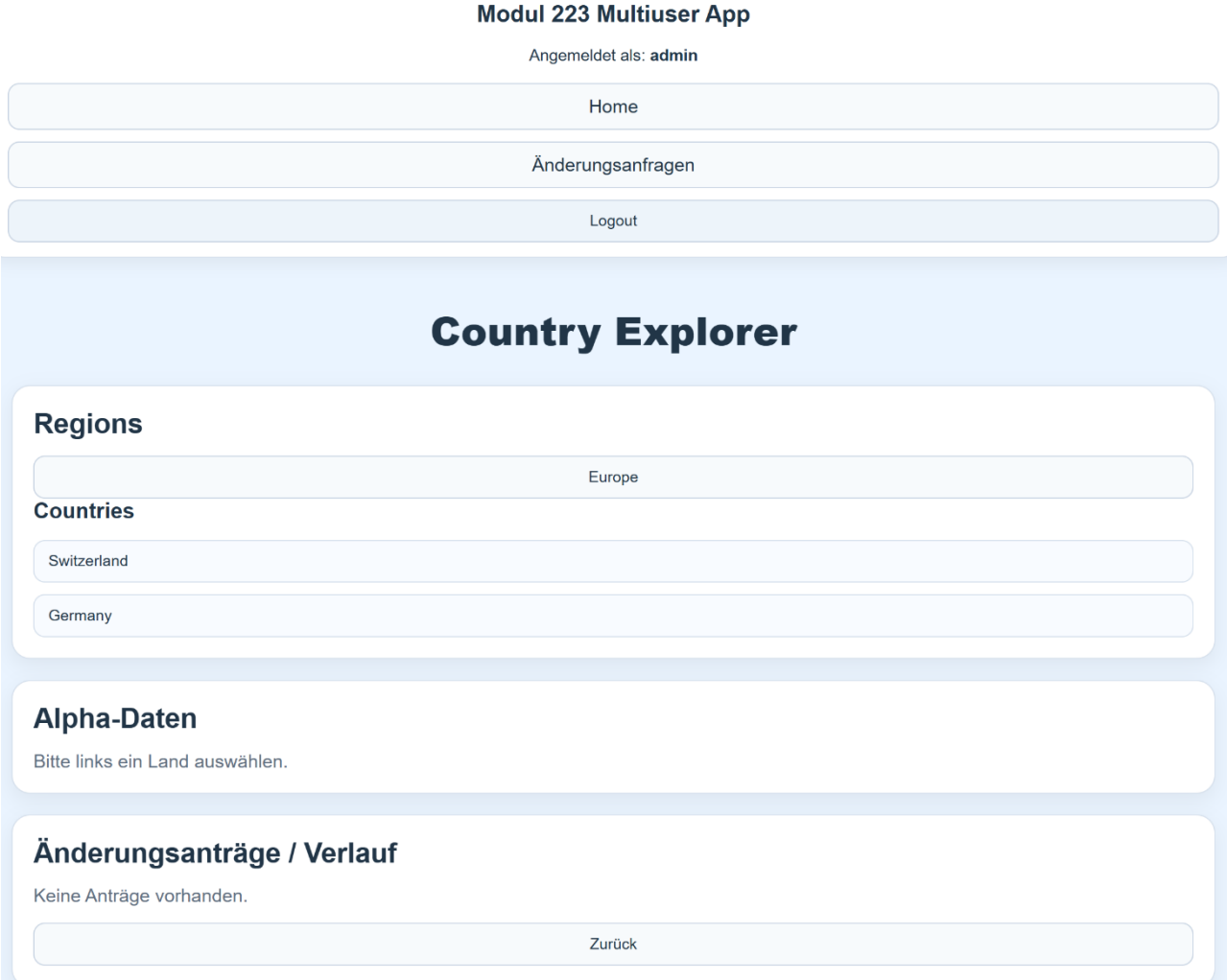
Dokumentation zum Projekt Modul 223



Autor:

Ajro Vullnet

Inhaltsverzeichnis

1 Einleitung ... 3
2 Projektübersicht ... 4
3 Projektziel und Abgrenzung ... 5
4 User Stories mit Akzeptanzkriterien ... 6
5 Arbeitsplan mit Aufwandschätzung ... 8
6 Arbeitsjournal je Block ... 11
7 Systemarchitektur ... 15
7.1 Gesamtarchitektur ... 15
7.2 Backend-Architektur ... 15
7.3 Frontend-Architektur ... 16
7.4 Datenfluss ... 16
8 Authentifizierung und Autorisierung ... 17
8.1 Login-Prozess ... 17
8.2 JWT-Verarbeitung ... 17
8.3 Rollenmodell und Zugriffsschutz ... 17
9 Sicherheitskonzept ... 18
9.1 Öffentliche und geschützte Bereiche ... 18
9.2 Schutz im Backend ... 18
9.3 Schutz im Frontend ... 18
9.4 Passwortschutz und Testbenutzer ... 19
9.5 Grenzen der Sicherheitslösung ... 19
10 Transaktionen ... 20
10.1 Fachlicher Hintergrund ... 20
10.2 Aktueller Projektstand ... 20
10.3 Begründung für den Einsatz ... 20
10.4 Architektonische Einordnung ... 20
11 Deployment und Startanleitung ... 21
11.1 Voraussetzungen ... 21
11.2 Datenbank vorbereiten ... 21
11.3 Backend starten ... 21
11.4 Frontend starten ... 21
11.5 Testzugänge ... 22
11.6 Erwartetes Verhalten nach dem Start ... 22
12 Testkonzept und Testprotokoll ... 23
12.1 Testziel ... 23
12.2 Teststrategie ... 23
12.3 Relevante Testfälle ... 24
12.4 Testprotokoll ... 25
12.5 Einordnung der Testergebnisse ... 25
13 Soll-Ist-Vergleich und Problemanalyse ... 26
13.1 Geplanter Soll-Zustand ... 26
13.2 Tatsächlicher Ist-Zustand ... 26
13.3 Abweichungen zwischen Planung und Umsetzung ... 26
13.4 Zentrale Probleme im Projektverlauf ... 27
13.5 Gesamteinschätzung ... 27
14 Verwendete Hilfsmittel ... 27
14.1 Entwicklungswerkzeuge ... 27
14.2 Hilfestellungen ... 27
15 Fazit ... 28

1 Einleitung

Im Modul 223 steht die Entwicklung einer Multiuser-Applikation im Mittelpunkt.

Ziel des vorliegenden Projekts war die Realisierung einer webbasierten Anwendung, in der öffentliche Informationen angezeigt und geschützte Funktionen nur für berechtigte Benutzer zugänglich gemacht werden.

Der Fokus lag dabei insbesondere auf der Umsetzung von Authentifizierung, Autorisierung und der fachlichen Trennung zwischen normalen Benutzern und Administratoren.

Die Grundlage des Projekts bestand nicht in einer vollständigen Neuentwicklung aller Bestandteile. Stattdessen baut das Projekt auf bereits vorhandenen Resultaten aus den Modulen 294 und 295 auf.

Das Frontend aus Modul 294 und das Backend aus Modul 295 mussten zunächst technisch zusammengeführt, an eine neue Datenbank angebunden und wieder in einen stabilen, lauffähigen Zustand gebracht werden.

Erst auf dieser Basis wurden die für Modul 223 relevanten Funktionen fachlich und technisch korrekt integriert.

Die Anwendung ermöglicht es, Länder und deren Alpha-Daten anzuzeigen. Zusätzlich können eingeloggte Benutzer Änderungsanträge für bestimmte Daten eines Landes erfassen.

Administratoren erhalten einen geschützten Bereich, in dem diese Änderungsanträge geprüft, genehmigt oder abgelehnt werden können. Die Anmeldung erfolgt über JWT, während der Zugriff auf geschützte Funktionen über Rollen gesteuert wird.

Die Projektdokumentation beschreibt den Aufbau, die Planung, die technische Umsetzung und die sicherheitsrelevanten Aspekte der Anwendung.

Zudem wird der Projektverlauf, die gewählten Architekturen, die Teststrategie sowie die Auswertung des erreichten Projektstands nachvollziehbar dargestellt.

2 Projektübersicht

Das Projekt wurde als webbasierte Multiuser-Anwendung mit React im Frontend und Spring Boot im Backend umgesetzt.

Die Anwendung trägt den Namen **Country Explorer** und dient dazu, ausgewählte Länderinformationen anzuzeigen sowie Änderungen an bestimmten Daten strukturiert über einen Antragsprozess zu verwalten.

Im öffentlichen Bereich können Länder ausgewählt und deren Alpha-Daten angezeigt werden.

Dazu gehören insbesondere Angaben wie Hauptstadt, Einwohnerzahl und Präsident. Diese Informationen stehen ohne Login zur Verfügung und bilden den lesenden Teil der Anwendung.

Zusätzlich besitzt die Anwendung einen geschützten Bereich für angemeldete Benutzer.

Eingeloggte Benutzer können zu einem ausgewählten Land einen Änderungsantrag erfassen.

Dabei wird ein bestimmtes Datenfeld ausgewählt und ein neuer Wert vorgeschlagen. Der Antrag wird anschliessend im Backend gespeichert und steht für die weitere Bearbeitung bereit.

Administratoren verfügen über erweiterte Rechte. Sie können vorhandene Änderungsanträge in einer separaten Admin-Ansicht anzeigen, prüfen und anschliessend genehmigen oder ablehnen.

Bei einer Genehmigung wird der vorgeschlagene Wert in die eigentlichen Länderdaten übernommen. Bei einer Ablehnung bleibt der bestehende Datenstand unverändert.
Die Authentifizierung wird über JWT umgesetzt.

Nach erfolgreichem Login erhält der Benutzer ein Token, das bei geschützten Anfragen mitgesendet wird. Die Autorisierung erfolgt rollenbasiert. Dadurch wird zwischen normalen Benutzern und Administratoren unterschieden. Öffentliche, geschützte und administrative Funktionen sind somit klar voneinander getrennt.

Fachlich konzentriert sich das Projekt auf einen bewusst eingegrenzten Funktionsumfang. Ziel war nicht eine möglichst grosse Anzahl von Features, sondern eine nachvollziehbare und funktionierende Multiuser-Anwendung mit klarer Rollenlogik, abgesicherten Endpunkten und einem einfachen, fachlich sinnvollen Änderungsprozess.

3 Projektziel und Abgrenzung

Ziel des Projekts war die Umsetzung einer kompakten Multiuser-Webanwendung, in der öffentliche Informationen und geschützte Funktionen sauber voneinander getrennt sind.

Im Mittelpunkt stand dabei nicht die möglichst umfangreiche fachliche Ausgestaltung, sondern die korrekte technische und fachliche Umsetzung eines klar abgegrenzten Anwendungsszenarios mit Benutzerrollen, Authentifizierung und kontrollierten Datenänderungen.

Die Anwendung sollte es ermöglichen, Länder und deren Alpha-Daten anzuzeigen, ohne dass dafür ein Login erforderlich ist. Zusätzlich sollte ein Antragsprozess umgesetzt werden, mit dem eingeloggte Benutzer Änderungsvorschläge für bestimmte Länderdaten erfassen können.

Diese Vorschläge sollten nicht direkt übernommen werden, sondern zuerst durch einen Administrator geprüft werden. Dadurch wird der eigentliche Multiuser-Charakter der Anwendung sichtbar: verschiedene Benutzergruppen arbeiten mit unterschiedlichen Rechten auf denselben Datenbestand.

Ein weiteres zentrales Ziel war die Integration der Modul-223-Themen in eine bereits bestehende Projektbasis.

Das Projekt wurde deshalb nicht als vollständige Neuentwicklung erstellt, sondern auf Basis vorhandener Resultate aus den Modulen 294 und 295 erweitert.

Daraus ergab sich als wesentliches Teilziel, Frontend und Backend technisch zusammenzuführen, mit einer neuen Datenbank lauffähig zu machen und die M223-Funktionen fachlich korrekt in dieses Gesamtsystem einzubinden.

Abgrenzung des Projekts
Der Funktionsumfang wurde bewusst reduziert, damit die Anwendung im Rahmen des Moduls überschaubar, nachvollziehbar und stabil bleibt. Der Schwerpunkt liegt auf folgenden Funktionen:

- Anzeige von Ländern und Alpha-Daten
- Login mit JWT
- rollenbasierte Unterscheidung zwischen Benutzer und Administrator
- Erfassung von Änderungsanträgen
- Admin-Bereich zur Bearbeitung von Änderungsanträgen

Die Abgrenzung war notwendig, um einen klaren, bewertbaren und funktionierenden Projektkern zu erhalten.

Auf diese Weise konnte sich das Projekt auf die im Modul zentralen Themen konzentrieren: Frontend- und Backend-Integration, Authentifizierung mit JWT, Autorisierung über Rollen sowie ein nachvollziehbarer geschützter Bearbeitungsprozess.

4 User Stories mit Akzeptanzkriterien

Die User Stories beschreiben die fachlichen Kernanforderungen der Anwendung aus Sicht der späteren Benutzergruppen.

Sie dienen dazu, den geplanten Funktionsumfang klar und verständlich zu formulieren. Zusätzlich werden zu jeder User Story Akzeptanzkriterien definiert. Diese legen fest, wann eine Anforderung als erfüllt gilt und bilden gleichzeitig eine Grundlage für spätere Tests und die Bewertung des Projekts.

Im vorliegenden Projekt wurden die User Stories bewusst kompakt gehalten und direkt auf den tatsächlich umgesetzten Funktionsumfang abgestimmt.

US-01 Länder und Alpha-Daten anzeigen

Als Besucher möchte ich auf der Startseite Länder auswählen und deren Alpha-Daten anzeigen können, damit ich die wichtigsten Informationen zu einem Land direkt abrufen kann.

Akzeptanzkriterien

- Beim Öffnen der Anwendung wird die Länderübersicht angezeigt.
- Ein Land kann ausgewählt werden.
- Nach der Auswahl werden die zugehörigen Alpha-Daten angezeigt.
- Die Anzeige funktioniert ohne Login.

US-02 Einloggen

Als registrierter Benutzer möchte ich mich mit Benutzername und Passwort anmelden können, damit ich geschützte Funktionen der Anwendung nutzen kann.

Akzeptanzkriterien

- Es gibt ein Login-Formular mit Benutzername und Passwort.
- Bei gültigen Zugangsdaten wird der Benutzer erfolgreich angemeldet.
- Nach erfolgreichem Login ist der Benutzerstatus in der Anwendung sichtbar.
- Bei ungültigen Zugangsdaten erscheint eine Fehlermeldung.

US-03 Änderungsantrag erstellen

Als eingeloggter Benutzer

möchte ich für ein Land einen Änderungsantrag erfassen können, damit veraltete oder fehlerhafte Alpha-Daten gemeldet werden können.

Akzeptanzkriterien

- Ein Änderungsantrag kann nur im eingeloggten Zustand erstellt werden.
- Vor dem Erfassen eines Antrags muss ein Land ausgewählt sein.
- Für den Antrag kann ein Feld ausgewählt und ein neuer Wert eingegeben werden.
- Nach dem Absenden wird der Antrag gespeichert und bestätigt.

US-04 Änderungsanträge als Admin bearbeiten

Als Administrator möchte ich Änderungsanträge anzeigen, genehmigen oder ablehnen können, damit nur geprüfte Änderungen in die Länderdaten übernommen werden.

Akzeptanzkriterien

- Der Admin-Bereich ist nur für Benutzer mit Admin-Rolle zugänglich.
- Vorhandene Änderungsanträge werden in einer Übersicht angezeigt.
- Ein Antrag kann genehmigt oder abgelehnt werden.
- Bei Genehmigung wird der neue Wert in den Länderdaten gespeichert.
- Bei Ablehnung wird keine Änderung an den Länderdaten vorgenommen.
- Nach der Entscheidung verschwindet der bearbeitete Antrag aus der Liste.

Diese vier User Stories decken den fachlichen Kern des Projekts vollständig ab.

Sie bilden die Grundlage für die technische Umsetzung, die spätere Testplanung sowie die Bewertung des Projekts im Hinblick auf die geforderten Multiuser-Funktionen.

5 Arbeitsplan mit Aufwandschätzung

Der Arbeitsplan dient dazu, die Umsetzung des Projekts in nachvollziehbare Arbeitspakete zu gliedern und den voraussichtlichen Aufwand realistisch einzuschätzen.

Da das Projekt nicht als vollständige Neuentwicklung entstanden ist, sondern auf bestehenden Resultaten aus Modul 294 und Modul 295 aufbaut, lag der Schwerpunkt insbesondere auf der Zusammenführung, Anpassung und Erweiterung der vorhandenen Projektbasis.

Die Planung wurde deshalb so aufgebaut, dass zuerst die bestehende Ausgangslage analysiert, danach die technische Basis zusammengeführt und anschliessend die eigentlichen M223-Funktionen integriert werden konnten.

Abschliessend wurden Stabilisierung, Dokumentation und Abgabevorbereitung berücksichtigt.

AP	Arbeitspaket	Inhalt	Ergebnis	Aufwand
AP-01	Projektanalyse und Scope-Definition	Bestehende Resultate aus M294 und M295 analysieren, Projektumfang eingrenzen und M223-Ziel definieren	Klar definierter Projektumfang	2 h
AP-02	Bestehende Projekte zusammenführen	Frontend aus M294 und Backend aus M295 in eine gemeinsame Projektbasis überführen	Gemeinsame lauffähige Grundstruktur	3 h
AP-03	Datenbank und Konfiguration anpassen	Neue Datenbank einrichten, Verbindungen anpassen und Anwendung technisch startfähig machen	Lauffähige Anwendung mit neuer Datenbank	3 h
AP-04	M223-Grundlage integrieren	Die im Unterricht erarbeiteten Bestandteile für M223 fachlich und technisch in das Projekt übernehmen	Korrekt eingebundene Multiuser-Basis	3 h
AP-05	Authentifizierung und Rollen einbinden	Login, JWT, Rollenmodell und geschützte Bereiche für Benutzer und Admin integrieren	Abgesicherte Multiuser-Zugriffe	4 h
AP-06	Änderungsantragsprozess umsetzen	Erfassung, Speicherung, Anzeige sowie Bearbeitung von Änderungsanträgen implementieren	Vollständiger Änderungsprozess	4 h
AP-07	Frontend- und Backend-Abgleich	Datenfluss, Routing, Rollenprüfung und Fehlerbehebung zwischen Frontend und Backend abstimmen	Stabile Gesamtfunktion	3 h
AP-08	Tests vorbereiten und durchführen	Sinnvolle Backend- und Frontend-Tests definieren, umsetzen und protokollieren	Testnachweise gemäss Bewertungskriterien	3 h
AP-09	Dokumentation und Abgabevorbereitung	Architektur, Sicherheit, Transaktionen, Arbeitsjournal, Auswertung und Startanleitung erstellen	Vollständige Projektdokumentation und Abgabe	4 h

Begründung der Planung

Die Planung orientiert sich am tatsächlichen Projektverlauf.

Zu Beginn stand nicht die Entwicklung einzelner neuer Funktionen im Vordergrund, sondern die Herstellung einer gemeinsamen technischen Basis.

Erst nachdem Frontend, Backend und Datenbank zusammengeführt und lauffähig gemacht worden waren, konnten die Multiuser-Funktionen aus Modul 223 sauber integriert werden. Der grösste inhaltliche Schwerpunkt lag auf der Authentifizierung mit JWT, der Rollenlogik sowie dem Änderungsantragsprozess zwischen Benutzer und Administrator.

Diese Arbeitspakete erhielten deshalb einen höheren geschätzten Aufwand als rein organisatorische oder beschreibende Aufgaben.

Zusätzlich wurde bewusst ein eigener Abschnitt für Tests sowie für Dokumentation und Abgabevorbereitung vorgesehen, da diese Punkte im Bewertungsraster explizit verlangt werden und einen wesentlichen Teil der Gesamtleistung ausmachen.

Der Arbeitsplan bildet damit nicht nur eine grobe Zeitplanung, sondern auch die fachliche Struktur des Projekts ab.

6 Arbeitsjournal je Block

Das Arbeitsjournal dokumentiert den Projektverlauf blockweise und zeigt auf, welche Arbeiten durchgeführt wurden, wie sie sich auf die geplanten Arbeitspakete beziehen und welche Probleme oder Besonderheiten dabei aufgetreten sind.
Da das Projekt auf bestehenden Resultaten aus Modul 294 und Modul 295 aufbaut, lag der Schwerpunkt nicht in einer vollständigen Neuentwicklung, sondern in der Integration, Anpassung und Erweiterung der vorhandenen Projektbasis.
Die folgenden Einträge orientieren sich am tatsächlichen Verlauf des Projekts und beschreiben die wesentlichen Fortschritte pro Block.

Block 1

Bezug Arbeitspakete: AP-01

Inhalt: Bestehende Resultate aus M294 und M295 analysiert. Den fachlichen Zielumfang für das M223-Projekt festgelegt und auf die zentralen Multiuser-Funktionen reduziert. Entscheidend war die Definition eines klaren und realistischen Projektscopes mit Länderanzeige, Login, Rollen, Änderungsanträgen und Admin-Bearbeitung.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Die vorhandenen Projektbestände enthielten zusätzliche Elemente, die für den eigentlichen M223-Umfang nicht relevant waren. Zu Beginn musste deshalb zuerst geklärt werden, welche Funktionen fachlich wirklich zum Projekt gehören und welche nur Altlasten aus früheren Modulen darstellen.

Block 2

Bezug Arbeitspakete: AP-01, AP-02

Inhalt: User Stories formuliert und Projektstruktur geplant. Das bestehende Frontend aus M294 und das bestehende Backend aus M295 fachlich miteinander abgestimmt. Die Zielanwendung wurde auf dieser Basis als gemeinsamer Country Explorer mit Multiuser-Funktionen definiert.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Frontend und Backend lagen nicht als bereits vorbereitetes M223-Gesamtprojekt vor, sondern mussten zunächst auf dieselbe fachliche Zielsetzung ausgerichtet werden. Dadurch war die frühe Planungsphase stärker von Strukturfragen geprägt als ursprünglich angenommen.

Block 3

Bezug Arbeitspakete: AP-02, AP-03

Inhalt: Die bestehenden Projektstände aus M294 und M295 technisch zusammengeführt. Neue Datenbankverbindung eingerichtet, Konfigurationen angepasst und das Projekt mit neuer Datenbasis wieder startfähig gemacht. Dabei wurden Projektstruktur, Datenbankzugriffe und Laufverhalten überprüft.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Nach der Zusammenführung lief das Projekt nicht sofort stabil. Verbindungsdaten, Projektkonfiguration und technische Abhängigkeiten mussten mehrfach überprüft und korrigiert werden, bis Frontend und Backend wieder gemeinsam funktionierten.

Block 4

Bezug Arbeitspakete: AP-04, AP-05

Inhalt: Die im Unterricht erarbeiteten Grundlagen zu Spring Security und JWT in das bestehende Projekt übernommen. Rollenmodell mit Benutzer und Administrator eingebunden. Geschützte und öffentliche Backend-Endpunkte fachlich voneinander getrennt.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Die Security-Struktur aus dem Unterricht konnte nicht unverändert übernommen werden, da sie an die bereits vorhandene Datenbasis und Projektstruktur angepasst werden musste. Insbesondere die Einbindung des Rollenmodells und die Trennung öffentlicher und geschützter Bereiche erforderten zusätzliche Abstimmung.

Block 5

Bezug Arbeitspakete: AP-05

Inhalt: Login-Endpunkt, Token-Erzeugung, Token-Prüfung und Security-Konfiguration im Backend vervollständigt. Die JWT-basierte Authentifizierung wurde so eingebunden, dass geschützte Funktionen nur mit gültigem Token ausgeführt werden können.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Es musste genau festgelegt werden, welche Endpunkte öffentlich erreichbar bleiben und welche authentifiziert sein müssen. Fehler in dieser Zuordnung hätten entweder die Länderanzeige blockiert oder geschützte Funktionen unzureichend abgesichert.

Block 6

Bezug Arbeitspakete: AP-04, AP-05

Inhalt: Die M223-Frontend-Grundlagen in das bestehende React-Projekt integriert. Navigation, Routing, Login-Ansicht und geschützte Bereiche im Frontend vorbereitet. Die bestehende Länder- und Detailansicht blieb dabei als fachliche Grundlage erhalten.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Die bestehende Frontend-Struktur aus M294 musste so angepasst werden, dass die Multiuser-Logik korrekt integriert werden konnte, ohne die bereits funktionierenden Ansichten für Länder und Alpha-Daten unnötig zu beschädigen.

Block 7

Bezug Arbeitspakete: AP-05, AP-06

Inhalt: Login im Frontend mit JWT-Anbindung umgesetzt. Der Benutzerstatus wurde über den Authentifizierungsaufbau zentral verwaltet und geschützte Funktionen an den Login-Zustand gekoppelt. Die Lösung wurde dabei vorlagennahe mit AuthContext und AdminRoute an den finalen Projektstand angepasst.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Ein zentrales Problem bestand darin, dass geschützte Requests anfänglich nicht korrekt mit einem Bearer-Token gesendet wurden. Dadurch kam es bei Änderungsanträgen und Admin-Zugriffen zu 401 Unauthorized. Die Header-Logik und der Authentifizierungsaufbau mussten deshalb überarbeitet und stabilisiert werden.

Block 8

Bezug Arbeitspakete: AP-06

Inhalt: Den Änderungsantragsprozess umgesetzt. Eingeloggte Benutzer können für ein ausgewähltes Land einen Antrag mit Feld und neuem Wert erfassen. Die Anträge werden im Backend gespeichert und für Administratoren im geschützten Bereich geladen.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Nach dem Absenden eines Antrags war dessen Anzeige im Admin-Bereich zunächst nicht korrekt. Die Ursache lag im Zusammenspiel aus Request-Aufbau im Frontend, JWT-Header und geschütztem Backend-Endpunkt. Erst nach der Korrektur der Request-Logik funktionierten Speicherung und Anzeige konsistent.

Block 9

Bezug Arbeitspakete: AP-06, AP-07

Inhalt: Die Admin-Funktionen vervollständigt. Änderungsanträge können angezeigt, genehmigt oder abgelehnt werden. Bei Genehmigung wird der neue Wert in die eigentlichen Länderdaten übernommen, bei Ablehnung bleibt der Datenstand unverändert.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Nach der Bearbeitung verschwanden Anträge zunächst nicht korrekt aus der Oberfläche oder Änderungen wurden nicht sofort sichtbar. Deshalb musste die Frontend-Logik so ergänzt werden, dass die Listenansicht aktualisiert und die Länderdetails nach einer Genehmigung neu geladen werden.

Block 10

Bezug Arbeitspakete: AP-07, AP-08, AP-09

Inhalt: Das Gesamtsystem stabilisiert, Routing bereinigt, die Authentifizierungsstruktur final eingebunden und der Projektstand für die Abgabe bereinigt. Dazu gehörten auch die Reduktion unnötiger Altbestandteile aus früheren Modulen sowie die inhaltliche Fertigstellung der Dokumentation mit Architektur, Sicherheit, Deployment, Tests und Auswertung.

Dauer: 4 Lektionen

Probleme / Besonderheiten: Kurz vor dem Abschluss musste mehrfach entschieden werden, welche Bereinigungen noch sinnvoll und risikoarm sind. Ziel war ein sauberer, stabiler und abgabefähiger Projektstand, ohne funktionierende Teile unnötig neu umzubauen.

7 Systemarchitektur

Die Anwendung ist als Fullstack-Webapplikation mit getrenntem Frontend und Backend aufgebaut. Das Frontend basiert auf React und ist für Darstellung, Navigation und Benutzerinteraktion zuständig. Das Backend wurde mit Spring Boot umgesetzt und übernimmt die Verarbeitung der HTTP-Anfragen, die Sicherheitslogik, die Rollenprüfung sowie den Datenzugriff auf die MySQL-Datenbank.
Die Kommunikation zwischen Frontend und Backend erfolgt über REST-Endpunkte mit JSON. Öffentliche Daten können ohne Login geladen werden. Geschützte Funktionen wie das Erstellen oder Bearbeiten von Änderungsanträgen setzen einen gültigen JWT-Token voraus.
Die Systemarchitektur ist in drei Hauptbereiche gegliedert:

- Frontend mit React
- Backend mit Spring Boot
- Persistenzschicht mit MySQL

Diese Trennung sorgt dafür, dass Oberfläche, Geschäftslogik und Datenhaltung klar voneinander abgegrenzt bleiben.

7.1 Gesamtarchitektur

Das Frontend übernimmt die Anzeige der Länder, die Navigation zwischen den Ansichten sowie die Benutzerinteraktion. Es sendet Requests an das Backend und verarbeitet die Antworten für die Darstellung in der Oberfläche.
Das Backend stellt die REST-Endpunkte bereit, prüft eingehende Anfragen, verarbeitet die fachliche Logik und greift über Repositories auf die Datenbank zu. Gleichzeitig übernimmt es die Sicherheitsprüfung für geschützte Funktionen.
Die MySQL-Datenbank speichert die persistenten Daten der Anwendung. Dazu gehören insbesondere Regionen, Länder, Benutzer, Rollen und Änderungsanträge.

7.2 Backend-Architektur

Das Backend ist in Controller, Security-Komponenten, Repositories und Datenmodelle gegliedert.
Die Controller bilden die Eintrittspunkte für HTTP-Anfragen. AuthController ist für Login und Registrierung zuständig. CountryController liefert die Länder- und Detaildaten. RegionController stellt Regionen bereit. ChangeRequestController verarbeitet Änderungsanträge sowie deren Bearbeitung im Admin-Bereich.
Die Sicherheitslogik ist von den fachlichen Endpunkten getrennt. SecurityConfiguration definiert öffentliche und geschützte Endpunkte. AuthTokenFilter liest Bearer-Tokens aus dem Header und validiert sie mit JwtUtils. UserDetailsServiceImpl lädt den zugehörigen Benutzer aus der Datenbank. UserDetailsImpl stellt die Benutzerdaten in der von Spring Security erwarteten Form bereit.
Der Datenzugriff erfolgt über Spring Data JPA Repositories. Im Projekt werden UserRepository, RoleRepository, CountryRepository, RegionRepository und ChangeRequestRepository verwendet.
Die fachlichen Daten werden über die Entitäten User, Role, Country, Region und ChangeRequest modelliert. Ein Benutzer besitzt Rollen, ein Land gehört zu genau einer Region und ein Änderungsantrag bezieht sich immer auf genau ein Land.

7.3 Frontend-Architektur

Das Frontend ist in Module, Komponenten, Context und Service-Dateien gegliedert.
main.jsx bildet den Einstiegspunkt der Anwendung. Dort werden BrowserRouter und AuthProvider eingebunden. App.jsx definiert die Routen. Layout.jsx liefert die Grundstruktur mit Navigation und eingebettetem Seiteninhalt.
Die zentrale Fachansicht ist Home.jsx. Dort werden Länder geladen, Details angezeigt, Änderungsanträge erfasst und im Admin-Fall bearbeitet. Login.jsx stellt die Login-Ansicht bereit. Navigation.jsx zeigt den aktuellen Benutzerstatus und die verfügbaren Navigationspunkte an.
Die Authentifizierungslogik ist im AuthContext.jsx gekapselt. auth.service.js übernimmt Login, Logout, Tokenzugriff und Speicherung im localStorage. Die eigentlichen Requests an die fachlichen Backend-Endpunkte sind in restcountries.js zusammengefasst.

7.4 Datenfluss

Der Datenfluss beginnt mit einer Benutzeraktion im Frontend. Diese löst entweder eine lokale Zustandsänderung oder einen Request an das Backend aus.
Beim Start der Anwendung lädt Home.jsx die Länder der aktiven Region. Wird ein Land ausgewählt, werden die zugehörigen Alpha-Daten vom Backend geladen und im Frontend angezeigt.
Beim Login sendet das Frontend Benutzername und Passwort an das Backend. Nach erfolgreicher Prüfung wird ein JWT zurückgegeben, im localStorage gespeichert und in den globalen Benutzerzustand übernommen.
Beim Erstellen eines Änderungsantrags sendet das Frontend die Antragsdaten zusammen mit dem Bearer-Token an das Backend. Das Backend speichert den Antrag mit Benutzername, Status und Zeitstempel.
Im Admin-Bereich werden offene Änderungsanträge geladen. Nach einer Genehmigung oder Ablehnung wird die Antragsliste aktualisiert. Bei einer Genehmigung werden zusätzlich die betroffenen Länderdaten neu geladen, damit die Änderung direkt sichtbar wird.

8 Authentifizierung und Autorisierung

Die Anwendung verwendet JWT-basierte Authentifizierung und rollenbasierte Autorisierung. Ziel ist es, geschützte Funktionen nur für berechtigte Benutzer zugänglich zu machen und gleichzeitig öffentliche Inhalte ohne Login bereitzustellen.

8.1 Login-Prozess

Der Login beginnt im Frontend mit der Eingabe von Benutzername und Passwort. Diese Daten werden an den Auth-Endpunkt des Backends gesendet. Im Backend prüft AuthController die Zugangsdaten mit dem AuthenticationManager. Bei erfolgreicher Prüfung wird ein JWT erzeugt und zusammen mit Benutzer-ID, Benutzername und Rollen an das Frontend zurückgegeben.

Das Frontend speichert diese Antwort im localStorage. Gleichzeitig wird der Benutzerzustand im AuthContext gesetzt. Dadurch kann die Anwendung direkt auf den erfolgreichen Login reagieren und geschützte Funktionen freischalten.

8.2 JWT-Verarbeitung

JwtUtils erzeugt und prüft Tokens im Backend. Beim Login wird ein neues Token mit Benutzername und Ablaufzeit erstellt. Bei geschützten Requests liest AuthTokenFilter den Bearer-Token aus dem Authorization-Header, validiert ihn und lädt anschliessend den zugehörigen Benutzer.

Im Frontend wird das JWT nach erfolgreichem Login gespeichert. auth.service.js stellt daraus den passenden Authorization-Header mit Bearer <token> bereit. restcountries.js verwendet diesen Header bei allen geschützten Requests.

Da die Anwendung zustandslos arbeitet, wird der Bearer-Token nicht in der Datenbank gespeichert, sondern bei jeder geschützten Anfrage erneut geprüft.

8.3 Rollenmodell und Zugriffsschutz

Im Projekt werden zwei Rollen verwendet:

- ROLE_USER
- ROLE_ADMIN

Ein Benutzer mit ROLE_USER darf Länder anzeigen und Änderungsanträge erfassen. Ein Benutzer mit ROLE_ADMIN darf zusätzlich den Admin-Bereich aufrufen und Änderungsanträge genehmigen oder ablehnen.

Die Rollen werden im Backend aus der Datenbank geladen und im Frontend zusätzlich zur Steuerung von Navigation und Routing ausgewertet.

9 Sicherheitskonzept

Das Sicherheitskonzept trennt konsequent zwischen öffentlichen und geschützten Funktionen. Die eigentliche technische Absicherung erfolgt im Backend. Das Frontend ergänzt diese Logik durch geschützte Navigation und kontrollierte Benutzerführung.

9.1 Öffentliche und geschützte Bereiche

Öffentlich zugänglich sind die Länderübersicht, die Alpha-Daten sowie die Login-Seite. Diese Inhalte können ohne Anmeldung verwendet werden.

Das Erstellen eines Änderungsantrags ist nur für eingeloggte Benutzer erlaubt. Dadurch ist jede Änderung eindeutig einem Benutzer zuordenbar.

Der Admin-Bereich ist ausschliesslich für Benutzer mit der Rolle ROLE_ADMIN freigegeben. Dort können offene Änderungsanträge angezeigt, genehmigt oder abgelehnt werden.

9.2 Schutz im Backend

SecurityConfiguration definiert, welche Endpunkte freigegeben sind und welche Authentifizierung benötigen. Öffentliche Länderdaten bleiben frei zugänglich. Geschützte Funktionen wie /change-requests benötigen einen gültigen Login.

AuthTokenFilter validiert eingehende Bearer-Tokens. AuthEntryPointJwt liefert bei unberechtigtem Zugriff einen 401 Unauthorized zurück. Dadurch bleibt die eigentliche Sicherheitsprüfung serverseitig zentral kontrolliert.

9.3 Schutz im Frontend

Im Frontend werden geschützte Bereiche zusätzlich über Routing abgesichert. AdminRoute.jsx verhindert den direkten Aufruf des Admin-Bereichs durch nicht eingeloggte oder nicht berechtigte Benutzer.

Zusätzlich reagiert restcountries.js auf 401 Unauthorized, indem der Benutzer ausgeloggt und auf die Login-Seite umgeleitet wird.

9.4 Passwortschutz und Testbenutzer

Passwörter werden mit BCryptPasswordEncoder gehasht gespeichert. Dadurch liegen sie nicht im Klartext in der Datenbank vor.

Für die Testbarkeit des Projekts werden beim Start automatisch zwei Benutzer erzeugt, sofern sie noch nicht existieren:

- user1
- admin

Beide Benutzer erhalten fest definierte Rollen und ermöglichen eine direkte Prüfung der Benutzer- und Administratorfunktionen.

9.5 Grenzen der Sicherheitslösung

Die umgesetzte Sicherheitslösung ist für den Modulrahmen fachlich passend und technisch nachvollziehbar. Nicht Bestandteil des Projekts sind unter anderem:

- Refresh-Token-Verwaltung
- Blacklisting ungültiger Tokens
- Passwort-Reset
- Selbstverwaltung von Benutzern
- detaillierte Sicherheitsprotokollierung

Für den gewählten Projektumfang ist die aktuelle Lösung ausreichend und korrekt.

10 Transaktionen

Im Projekt werden Datenänderungen insbesondere bei der Bearbeitung von Änderungsanträgen relevant. Dabei muss sichergestellt werden, dass zusammengehörende Änderungen konsistent verarbeitet werden. Wird ein Änderungsantrag genehmigt, betrifft dies fachlich nicht nur den Antrag selbst, sondern auch die eigentlichen Länderdaten.

10.1 Fachlicher Hintergrund

Beim Genehmigen eines Änderungsantrags passieren logisch zwei Schritte:

1. Der neue Wert wird in das betroffene Land übernommen.
2. Der bearbeitete Änderungsantrag wird entfernt.

Diese beiden Schritte gehören fachlich zusammen. Es wäre unerwünscht, wenn beispielsweise das Land geändert würde, der Antrag aber bestehen bliebe, oder umgekehrt der Antrag gelöscht würde, ohne dass die eigentliche Änderung gespeichert wurde.

10.2 Aktueller Projektstand

Im aktuellen Projekt werden diese zusammengehörenden Datenänderungen innerhalb des Controllers verarbeitet. Dabei wird zuerst das Land angepasst und gespeichert. Danach wird der Änderungsantrag gelöscht. Im laufenden Projekt funktioniert dieser Ablauf fachlich korrekt.

Für grössere oder produktionsnähere Anwendungen wäre es jedoch sauberer, diese Logik in eine eigene Service-Schicht auszulagern und dort explizit mit @Transactional abzusichern. Dadurch würde sichergestellt, dass die zusammengehörenden Datenänderungen atomar verarbeitet werden.

10.3 Begründung für den Einsatz

Transaktionen sind in diesem Projekt insbesondere deshalb relevant, weil Genehmigungen und Ablehnungen keine rein lesenden Vorgänge sind, sondern mehrere zusammenhängende Schreiboperationen auslösen können. Die fachliche Einheit dieser Operationen rechtfertigt den Einsatz transaktionaler Verarbeitung.

Die gewählte Lösung ist für den aktuellen Projektumfang ausreichend, da die Änderungsvorgänge einfach und überschaubar bleiben. Gleichzeitig zeigt das Projekt klar das Verständnis dafür, an welchen Stellen Transaktionen fachlich notwendig sind und wie sie in einer weiter ausgebauten Architektur sinnvoll eingesetzt würden.

10.4 Architektonische Einordnung

Aus architektonischer Sicht wäre die saubere Langfristlösung der Einsatz einer Service-Klasse zwischen Controller und Repository. Dort würde die Geschäftslogik des Änderungsprozesses gebündelt und mit @Transactional abgesichert. Da der Projektumfang bewusst kompakt gehalten wurde, wurde auf eine zusätzliche Service-Schicht verzichtet. Die fachliche Begründung für Transaktionen bleibt dennoch relevant und ist im Projektkontext klar erkennbar.

11 Deployment und Startanleitung

Damit das Projekt nach Anleitung lauffähig und testbar ist, müssen Backend, Frontend und Datenbank korrekt vorbereitet werden. Die Anwendung wurde so aufgebaut, dass nach dem Anlegen einer leeren Datenbank möglichst viele Schritte automatisch erfolgen.

11.1 Voraussetzungen

Für den Start der Anwendung werden benötigt:

- Java
- Maven
- Node.js und npm
- MySQL
- eine leere Datenbank mit dem Namen m223_country_explorer

Zusätzlich müssen die Datenbank-Zugangsdaten in der Datei application.properties korrekt gesetzt sein.

11.2 Datenbank vorbereiten

Zuerst wird in MySQL eine leere Datenbank angelegt. Die Tabellen müssen nicht manuell erstellt werden. Diese entstehen beim Start des Backends automatisch durch JPA beziehungsweise Hibernate.

Die Datei data.sql fügt anschliessend die fachlichen Basisdaten ein, insbesondere:

- Region Europe
- Land Switzerland
- Land Germany

Zusätzlich erzeugt eine Startinitialisierung automatisch die benötigten Rollen und Testbenutzer, sofern sie noch nicht vorhanden sind.

11.3 Backend starten

Das Backend wird im Projektordner mit Maven gestartet.

```
cd C:\LB-Projekt-M223_Vullnet_Ajro\Backend\Country_Explorer_Backend
mvn spring-boot:run
```

Beim Start werden:

- die Tabellen erzeugt
- die Basisdaten eingefügt
- die Rollen ROLE_USER und ROLE_ADMIN angelegt
- die Benutzer user1 und admin erstellt

11.4 Frontend starten

Das Frontend wird im Projektordner mit npm gestartet.

```
cd C:\LB-Projekt-M223_Vullnet_Ajro\Frontend\Country_Explorer_Frontend
npm run dev
```

Danach ist die Anwendung standardmässig unter der lokalen Vite-Adresse erreichbar.

11.5 Testzugänge

Für die Prüfung und Demonstration der Anwendung stehen folgende Testzugänge zur Verfügung:

- Benutzer: user1
  Passwort: 123456
- Administrator: admin
  Passwort: 123456

11.6 Erwartetes Verhalten nach dem Start

Nach erfolgreichem Start soll Folgendes möglich sein:

- Öffentliche Anzeige von Ländern und Alpha-Daten ohne Login
- Login als normaler Benutzer
- Erstellen eines Änderungsantrags als Benutzer
- Login als Administrator
- Anzeige, Genehmigung und Ablehnung von Änderungsanträgen im Admin-Bereich

12 Testkonzept und Testprotokoll

Zur Qualitätssicherung des Projekts werden die wichtigsten fachlichen und technischen Funktionen systematisch geprüft. Ziel der Tests ist es, sicherzustellen, dass die Anwendung nicht nur grundsätzlich startet, sondern die zentralen Multiuser-Funktionen korrekt und nachvollziehbar ausführt.

Im Fokus stehen dabei insbesondere folgende Bereiche:

- Login und JWT-basierte Authentifizierung
- rollenbasierter Zugriffsschutz
- Erfassung von Änderungsanträgen
- Bearbeitung von Änderungsanträgen im Admin-Bereich
- korrekte Anzeige und Aktualisierung von Länderdaten

12.1 Testziel

Das Testkonzept verfolgt das Ziel, die wesentlichen Funktionen des Projekts funktional und sicherheitstechnisch zu überprüfen. Besonders wichtig ist dabei, dass sich die Anwendung für unterschiedliche Benutzerrollen korrekt verhält und geschützte Funktionen nicht unberechtigt ausgeführt werden können.

12.2 Teststrategie

Die Teststrategie konzentriert sich auf die fachlich und technisch wichtigsten Abläufe der Anwendung. Geprüft werden sowohl das Verhalten des Backends als auch die Benutzerabläufe im Frontend.

Im Mittelpunkt stehen insbesondere folgende Punkte:

- korrekter Login mit JWT
- Zugriffsschutz für geschützte Funktionen
- Erfassung von Änderungsanträgen
- Bearbeitung von Änderungsanträgen im Admin-Bereich
- korrekte Aktualisierung der Länderdaten nach einer Entscheidung

Die dokumentierten Testfälle bilden damit die zentralen Multiuser-Funktionen des Projekts ab und prüfen, ob Benutzer- und Administratorabläufe im Gesamtsystem korrekt funktionieren.

12.3 Relevante Testfälle

Die folgenden Testfälle decken den Kern des Projekts ab.

Testfall 1 – Login mit gültigen Zugangsdaten

Ziel: Prüfen, ob ein Benutzer mit korrektem Benutzernamen und Passwort erfolgreich angemeldet werden kann.
Erwartetes Ergebnis: Das Backend liefert ein JWT zurück. Im Frontend wird der Benutzer als angemeldet angezeigt.

Testfall 2 – Geschützter Zugriff ohne Login

Ziel: Prüfen, ob ein nicht eingeloggter Benutzer keine geschützten Funktionen ausführen kann.
Erwartetes Ergebnis: Der Zugriff auf den Admin-Bereich wird blockiert und auf die Login-Seite umgeleitet.

Testfall 3 – Änderungsantrag als Benutzer erstellen

Ziel: Prüfen, ob ein eingeloggter Benutzer einen Änderungsantrag erfassen kann.
Erwartetes Ergebnis: Der Antrag wird gespeichert und steht im Admin-Bereich zur Verfügung.

Testfall 4 – Änderungsantrag als Admin genehmigen

Ziel: Prüfen, ob ein Administrator einen offenen Antrag genehmigen kann.
Erwartetes Ergebnis: Der geänderte Wert wird im Land gespeichert und der Antrag wird aus der Liste entfernt.

Testfall 5 – Änderungsantrag als Admin ablehnen

Ziel: Prüfen, ob ein Administrator einen offenen Antrag ablehnen kann.
Erwartetes Ergebnis: Der Antrag verschwindet aus der Liste, die Länderdaten bleiben unverändert.

12.4 Testprotokoll

Im Projekt wurden die zentralen Funktionen anhand der definierten Testfälle geprüft.

Testfall	Beschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
T-01	Login mit gültigen Zugangsdaten	JWT wird erzeugt, Benutzer ist angemeldet	Login funktionierte mit user1 und admin korrekt	Bestanden
T-02	Zugriff auf Admin-Bereich ohne Login	Weiterleitung auf Login-Seite	Weiterleitung erfolgte korrekt	Bestanden
T-03	Änderungsantrag als Benutzer erstellen	Antrag wird gespeichert	Antrag wurde im Backend gespeichert und im Admin-Bereich sichtbar	Bestanden
T-04	Änderungsantrag genehmigen	Wert wird übernommen, Antrag verschwindet	Neuer Wert wurde gespeichert, Antrag entfernt	Bestanden
T-05	Änderungsantrag ablehnen	Antrag verschwindet, keine Datenänderung	Antrag entfernt, Länderdaten unverändert	Bestanden

12.5 Einordnung der Testergebnisse

Die durchgeführten Tests zeigen, dass die wesentlichen Funktionen der Anwendung im aktuellen Projektstand korrekt arbeiten. Insbesondere Login, Rollenprüfung, Änderungsanträge und Admin-Bearbeitung konnten erfolgreich nachvollzogen werden.

Damit ist die zentrale Multiuser-Logik des Projekts funktional abgesichert.

13 Soll-Ist-Vergleich und Problemanalyse

Im Soll-Ist-Vergleich wird der ursprünglich geplante Projektumfang mit dem tatsächlich erreichten Stand verglichen. Ziel ist es, die Umsetzung realistisch einzuordnen und wichtige Probleme aus dem Projektverlauf transparent zu dokumentieren.

13.1 Geplanter Soll-Zustand

Geplant war eine kompakte Multiuser-Webanwendung mit folgenden Kernfunktionen:

- Länder und Alpha-Daten anzeigen
- Login mit JWT
- Unterscheidung zwischen Benutzer und Administrator
- Änderungsanträge für eingeloggte Benutzer
- Admin-Bereich zum Prüfen, Genehmigen und Ablehnen von Anträgen
- klare Trennung zwischen öffentlichen und geschützten Funktionen

Zusätzlich sollte das Projekt in einer sauberen Dokumentation beschrieben und für die Abgabe nachvollziehbar aufbereitet werden.

13.2 Tatsächlicher Ist-Zustand

Der aktuelle Projektstand erfüllt die wesentlichen Kernanforderungen des definierten Umfangs.

Umgesetzt wurden:

- öffentliche Anzeige von Ländern und Alpha-Daten
- JWT-basierter Login
- Rollenmodell mit ROLE_USER und ROLE_ADMIN
- Erfassung von Änderungsanträgen
- Admin-Bereich zur Bearbeitung der Anträge
- Aktualisierung der Länderdaten bei Genehmigung
- automatische Erstellung der Testbenutzer user1 und admin
- vereinfachter und bereinigter Projektstand ohne unnötige Altbestandteile

13.3 Abweichungen zwischen Planung und Umsetzung

Eine zentrale Abweichung bestand darin, dass das Projekt nicht als vollständige Neuentwicklung umgesetzt wurde, sondern auf vorhandenen Resultaten aus M294 und M295 aufbaute. Dadurch lag ein erheblicher Teil des tatsächlichen Aufwands in der technischen Zusammenführung, Bereinigung und Stabilisierung bereits vorhandener Projektteile.

Zudem mussten im Verlauf des Projekts mehrere Altlasten aus früheren Ständen entfernt oder reduziert werden, damit der aktuelle M223-Umfang sauber und nachvollziehbar bleibt. Dazu gehörten insbesondere nicht mehr benötigte Felder, Rollen und zusätzliche fachliche Bestandteile, die nicht mehr zum Zielumfang gehörten.

13.4 Zentrale Probleme im Projektverlauf

Während der Umsetzung traten mehrere Probleme auf, die schrittweise behoben werden mussten.

Ein wesentliches Problem war die Zusammenführung von Frontend und Backend aus unterschiedlichen Projektständen. Die Anwendung war nicht von Anfang an als direkt lauffähiges M223-Gesamtprojekt vorbereitet und musste zuerst technisch vereinheitlicht werden.

Ein weiteres Problem lag in der Authentifizierung. Geschützte Requests wurden anfänglich nicht korrekt mit Bearer-Token gesendet, wodurch geschützte Funktionen nicht funktionierten. Erst nach der Korrektur des Auth-Aufbaus in Frontend und Backend lief die JWT-Logik stabil.

Zusätzlich mussten bestehende Altstrukturen im Datenmodell reduziert werden, damit nur noch die wirklich benötigten Bestandteile für das aktuelle Projekt bestehen bleiben. Dadurch wurde das Projekt fachlich klarer und technisch besser auf den effektiven Umfang abgestimmt.

13.5 Gesamteinschätzung

Insgesamt wurde der geplante Kern des Projekts erreicht. Die Anwendung erfüllt die fachlichen Anforderungen einer kompakten Multiuser-Webanwendung mit JWT, Rollen, Änderungsanträgen und Admin-Bearbeitung. Der tatsächliche Projektverlauf war stärker von Integrations- und Bereinigungsarbeit geprägt als eine reine Neuentwicklung, was jedoch für den gewählten Ausgangszustand nachvollziehbar ist.

14 Verwendete Hilfsmittel

Während der Umsetzung wurden verschiedene Werkzeuge und Hilfsmittel eingesetzt, um Entwicklung, Test und Dokumentation effizient durchzuführen.

14.1 Entwicklungswerkzeuge

Für das Projekt wurden insbesondere folgende Werkzeuge verwendet:

- Visual Studio Code für das Frontend
- Eclipse beziehungsweise eine Java-Entwicklungsumgebung für das Backend
- MySQL Workbench für die Datenbankverwaltung
- Maven zum Starten und Verwalten des Backends
- npm und Vite zum Starten des Frontends
- Git und GitHub zur Versionsverwaltung

14.2 Hilfestellungen

Zur Unterstützung bei Konzeption, Strukturierung, Problemanalyse und Formulierung der Dokumentation wurden digitale Hilfsmittel verwendet. Dazu gehörten insbesondere allgemeine Internetrecherchen sowie ChatGPT zur inhaltlichen Strukturierung, Erklärung technischer Zusammenhänge und sprachlichen Ausarbeitung einzelner Dokumentationsabschnitte.

Die fachliche Umsetzung, Projektanpassung und technische Integration wurden anschliessend projektbezogen geprüft, eingegrenzt und auf den tatsächlichen Projektstand angepasst.

15 Fazit

Im Projekt wurde eine kompakte Multiuser-Webanwendung auf Basis von React, Spring Boot und MySQL umgesetzt. Im Zentrum stand die Integration der M223-Themen in eine bereits bestehende Projektbasis aus früheren Modulen. Dadurch lag der Schwerpunkt nicht nur auf neuen Funktionen, sondern auch auf der technischen Zusammenführung, Bereinigung und Stabilisierung des Gesamtsystems.

Die Anwendung erfüllt die wesentlichen Anforderungen des gewählten Umfangs. Länder und Alpha-Daten können öffentlich angezeigt werden. Eingeloggte Benutzer können Änderungsanträge erfassen. Administratoren können diese Anträge prüfen, genehmigen oder ablehnen. Die Authentifizierung erfolgt über JWT, während die Autorisierung rollenbasiert umgesetzt wurde.

Besonders deutlich wurde im Projekt, dass Multiuser-Anwendungen nicht nur aus Benutzeroberfläche und Datenbankzugriff bestehen, sondern vor allem aus dem sauberen Zusammenspiel von Frontend, Backend, Sicherheitslogik und Datenfluss. Die gewählte Architektur und die umgesetzte Rollenlogik bilden dafür eine nachvollziehbare und für den Modulrahmen geeignete Grundlage.

Insgesamt entstand eine funktionierende und klar abgegrenzte Anwendung, die die zentralen Themen des Moduls 223 praxisnah umsetzt.